



Llama 3.2 1B

FINE-TUNING DE LLM PARA EXTRAER ESTRUCTURADOS

NATURAL LANGUAGE TO JSON



Quiero vender...
I want to sell...
Quiero vender...
I want to sell...

QLoRA

```
{  
  "product": "...",  
  "price": ...  
}
```



LLAMA-3.2-1B



QLORA



HUMAN-TECH
BRIDGE

DESARROLLADO POR
IVANA SÁNCHEZ PÉREZ

Memoria Técnica: Fine-Tuning de LLM para Extracción de Datos Estructurados (Natural Language to JSON)

Tecnología: Llama-3.2-1B / QLoRA

Entorno: Google Colab (GPU NVIDIA Tesla T4)

1. Introducción y Concepto del Proyecto

El presente proyecto consiste en el desarrollo de un Conversor de Lenguaje Natural a JSON estructurado. Esta tecnología actúa como un puente de comunicación entre la ambigüedad del lenguaje humano y la precisión requerida por los sistemas informáticos.

¿En qué consiste la idea?

Un sistema de este tipo toma una instrucción escrita de manera informal por un usuario y la transforma automáticamente en un objeto JSON con una estructura definida. Este proceso implica tres fases críticas:

1. **Entrada:** Recepción de lenguaje natural (ej. "Quiero vender un iPhone 15 por 1200€").
2. **Procesamiento:** Un modelo de lenguaje (LLM), previamente entrenado, interpreta la intención y extrae las entidades clave.
3. **Salida:** Generación de un archivo JSON ordenado, listo para ser consumido por una API o base de datos.

Valor e Innovación

Esta solución es innovadora, ágil y escalable. Permite automatizar flujos de trabajo en sectores como la banca (transferencias por voz), e-commerce (filtros de búsqueda inteligentes) e IoT (rutinas domóticas), eliminando la necesidad de que un humano traduzca manualmente peticiones textuales a código de programación.

2. Arquitectura Técnica y Modelo Base

Para la implementación, se seleccionó el modelo Llama-3.2-1B-Instruct. Aunque inicialmente se consideró la versión de 8B, se optó por la de 1B para

garantizar la viabilidad del entrenamiento en entornos de hardware con recursos limitados (GPU T4), asegurando una latencia menor y una eficiencia energética superior sin sacrificar la precisión en la generación de JSON.

Optimización con QLoRA

Se ha empleado la técnica QLoRA (Quantized Low-Rank Adaptation), que permite realizar un ajuste fino extremadamente eficiente mediante:

- **Cuantización de 4 bits:** Reducción del peso del modelo para minimizar el uso de VRAM.
- **Adaptadores LoRA:** En lugar de reentrenar los miles de millones de parámetros del modelo, se entrenan pequeñas matrices de rango bajo ($r=16$) que actúan como "especialistas" en la tarea de conversión.

3. Configuración del Entrenamiento (Hiperparámetros)

Siguiendo los requisitos de la práctica y las optimizaciones de memoria necesarias, se configuraron los siguientes parámetros de entrenamiento:

Parámetro	Valor	Descripción
Learning Rate	2×10^{-4}	Tasa de aprendizaje óptima para evitar el olvido catastrófico.
Epochs	4.64	Número de pasadas completas por el dataset.
Rank (r)	16	Complejidad de las matrices de adaptación LoRA.
LoRA Alpha	32	Factor de escalado de la importancia del adaptador.
Batch Size	1 (con acumulación de 4)	Configuración para maximizar la estabilidad en GPU T4.
Max Steps	60	Pasos totales de optimización realizados.

4. Proceso de Ajuste Fino (Fine-Tuning)

El modelo fue entrenado con un dataset específico diseñado en formato Alpaca, que relaciona descripciones de productos de venta con su correspondiente representación en JSON.

Durante el proceso, se monitorizó la métrica de Training Loss. El entrenamiento comenzó con una pérdida elevada que descendió de manera constante hasta alcanzar un valor final de 0.013400 en el paso 60. Esta curva de aprendizaje descendente confirma que la red neuronal ha convergido satisfactoriamente y ha "entendido" la estructura sintáctica del JSON.

5. Evaluación y Resultados

Para validar el sistema, se realizó una prueba de inferencia con una frase totalmente nueva para el modelo:

Entrada: *"I want to buy a Sony PlayStation 5 for 500 dollars in the gaming category."*

Resultado Obtenido:

JSON

```
{  
  "product": "Sony PlayStation 5",  
  "price": 500,  
  "category": "gaming"  
}
```

Conclusión de la prueba: El modelo demuestra una capacidad perfecta de extracción de entidades (producto, precio y categoría) y una adherencia absoluta al formato JSON, sin incluir texto adicional innecesario o errores de sintaxis (como comas faltantes).

6. Reflexión Crítica

El desarrollo de este proyecto demuestra que, mediante el uso de técnicas modernas de **PEFT (Parameter-Efficient Fine-Tuning)** como QLoRA, es posible especializar modelos de lenguaje pequeños para tareas complejas de estructuración de datos.

A pesar de los retos técnicos iniciales relacionados con la gestión de memoria VRAM, la transición hacia un modelo de 1B y el ajuste de la longitud de secuencia permitieron obtener un conversor robusto, rápido y listo para ser integrado en aplicaciones de producción que requieran traducir el "español/inglés" al "idioma de las máquinas".

7. Disponibilidad y Despliegue

El modelo final, incluyendo los adaptadores LoRA entrenados, ha sido publicado en el Hugging Face Hub. Esto permite que cualquier desarrollador pueda descargar los pesos y utilizarlos en un entorno local o de producción de forma inmediata.

Enlace al repositorio: <https://huggingface.co/ivanasp/Llama-3.2-1B-JSON-Extractor>

Instrucciones de uso: El modelo requiere la librería `unsloth` o `peft` para cargar los adaptadores sobre el modelo base de Llama 3.2. Se recomienda el uso de la plantilla `alpaca_prompt` definida en esta memoria para obtener resultados óptimos.

8. Stack Tecnológico y Dependencias

Para asegurar la reproducibilidad del entrenamiento y la correcta inferencia del modelo, se han utilizado las siguientes librerías de última generación en el ecosistema de *Deep Learning*:

- **Unsloth:** Framework de optimización que permite un entrenamiento **2x más rápido** y reduce el consumo de memoria en un **70%**, fundamental para la viabilidad en la GPU Tesla T4.
- **Hugging Face transformers & peft:** Bibliotecas estándar para la gestión de modelos *Pre-trained* y la implementación de **PEFT** (Parameter-Efficient Fine-Tuning).
- **TRL (Transformer Reinforcement Learning):** Utilizada específicamente para la clase `SFTTrainer`, que facilita el entrenamiento supervisado por pasos.
- **Bitsandbytes:** Motor de cuantización encargado de gestionar la precisión de **4 bits**, permitiendo cargar el modelo Llama-3.2 con un uso de VRAM mínimo.